Technical Test

Engineering Intern — AI & Fintech

DURATION	FORMAT
72 hours from receipt	**Take-home, async, self-paced**
SUBMISSION	TOTAL MARKS
GitHub repo + short Loom/video walkthrough	**100 points across 4 tasks**

CONTEXT — WHAT TIMECELL.AI IS BUILDING

Timecell is an AI-powered wealth management platform for high-net-worth Indian families. We connect to market data, compute portfolio risk metrics (crash survival, temperature, position sizing), and surface intelligent recommendations — all powered by Claude as the AI engine.

This test mirrors the kind of work you will actually do. We value clear thinking, clean code, and the ability to use AI tools effectively — not memorised algorithms.

00 BEFORE YOU BEGIN — RULES & NOTES

WHAT IS ALLOWED

You may use any Python libraries. You may use Claude, Copilot, Cursor, or any AI coding tool. You are encouraged to. We want to see how you work with AI, not whether you can write code without it.

WHAT IS NOT ALLOWED

Do not submit code you did not write or understand. Do not copy solutions from others. If we ask you to explain your code in the follow-up call, you must be able to walk us through every line.

Additional notes:

- All tasks use Python 3.10+. No other language accepted.
- Use a public GitHub repository. Name it: timecell-intern-<your-name>.
- Include a README.md explaining your approach for each task.
- A 3–5 minute Loom (or any screen recording) walking through your solution is mandatory — not optional.
- You do not need to complete all 4 tasks. A well-reasoned partial submission beats a rushed complete one.
- If you get stuck, document your thinking in the README. We evaluate problem-solving approach, not just working code.

01 PORTFOLIO RISK CALCULATOR

Task 01 **Portfolio Risk Calculator**	**30 pts** weight	**Medium** difficulty

The Problem

You are given a portfolio of assets. Your job is to compute key risk metrics that a wealth manager would use to assess whether the portfolio is safe.

Input

A Python dictionary representing a portfolio:

```
portfolio = {
    "total_value_inr": 10_000_000,   # 1 Crore INR
    "monthly_expenses_inr": 80_000,
    "assets": [
        {"name": "BTC",    "allocation_pct": 30, "expected_crash_pct": -80},
        {"name": "NIFTY50","allocation_pct": 40, "expected_crash_pct": -40},
        {"name": "GOLD",   "allocation_pct": 20, "expected_crash_pct": -15},
        {"name": "CASH",   "allocation_pct": 10, "expected_crash_pct":   0},
    ]
}
```

What to Compute

Write a Python function compute_risk_metrics(portfolio) that returns a dictionary with:

- → post_crash_value — total portfolio value after the crash scenario
- → runway_months — how many months the post-crash portfolio can cover expenses
- → ruin_test — 'PASS' if runway > 12 months, 'FAIL' otherwise
- → largest_risk_asset — name of the asset with highest (allocation × crash magnitude)
- → concentration_warning — True if any single asset > 40% of portfolio

Bonus (Optional)

- → Add a second scenario: moderate crash (each asset loses 50% of its expected crash magnitude). Show both scenarios side by side.
- → Visualise the allocation breakdown as a simple CLI bar chart (no external plotting libraries).

What We Are Evaluating

DIMENSION	WEIGHT	WHAT WE LOOK FOR
Correctness	12 pts	Math is right, edge cases handled (zero allocation, 100% cash, etc.)
Code clarity	10 pts	Clean functions, good variable names, no spaghetti
AI usage	8 pts	Did you use Claude/Copilot well? Mention it in README.

---

02 LIVE MARKET DATA FETCH

Task 02 **Live Market Data Fetch**	20 pts weight	Easy difficulty

The Problem

Any wealth management product lives or dies by the quality of its data pipeline. Build a simple data fetcher.

What to Build

Write a Python script that:

- → Fetches the current price of any 3 assets of your choice — at least one must be a stock or index (e.g. NIFTY50, Sensex, any NSE-listed stock), and at least one must be a crypto asset (e.g. BTC, ETH).
- → Prints a clean, formatted table to the terminal with: asset name, current price, currency, and timestamp of fetch.
- → Handles errors gracefully — if a fetch fails, log the error and continue with the other assets.
- → Uses only free, public APIs — no paid API keys required. (Suggestions: Yahoo Finance via yfinance, CoinGecko for crypto, Alpha Vantage free tier.)

Expected Terminal Output (example)

```
Asset Prices — fetched at 2025-06-01 10:32:15 IST

| Asset | Price | Currency |
|---|---|---|
| BTC | 62,341.20 | USD |
| NIFTY | 22,541.80 | INR |
| GOLD | 7,312.00 | INR/g |
```

What We Are Evaluating

DIMENSION	WEIGHT	WHAT WE LOOK FOR
Working fetch	12 pts	Actually runs, fetches real data, table renders correctly
Error handling	8 pts	Doesn't crash when one API is down. Logs failures cleanly.
Code style	5 pts	Is it the kind of code you'd be proud to commit?

03 AI-POWERED PORTFOLIO EXPLAINER

Task 03 **AI-Powered Portfolio Explainer**	30 pts weight	Hard difficulty

The Problem

This is the most important task. It tests whether you can build AI-powered features — the core skill for this internship.

What to Build

Using any LLM API (OpenAI, Google Gemini, or Anthropic Claude), write a Python script that takes a portfolio (same structure as Task 1) and generates a plain-English explanation of the portfolio's risk — written in the tone of a friendly but honest financial advisor talking to a non-expert client.

The output should include:

- → A 3–4 sentence plain-English summary of the portfolio's risk level
- → One specific thing the investor is doing well
- → One specific thing the investor should consider changing, and why
- → A one-line verdict: 'Aggressive', 'Balanced', or 'Conservative'

LLM API SETUP — USE ANY PROVIDER

Use any LLM API you have access to — OpenAI (GPT-4o), Google Gemini, or Anthropic Claude are all equally valid. Pick whichever you already have a key for, or sign up for free credits: OpenAI at platform.openai.com, Gemini at aistudio.google.com, or Anthropic at console.anthropic.com. Document which API you used and why. The prompt engineering matters far more than which provider you choose.

Requirements

- → The prompt you send to the LLM must be part of your code — show us how you engineered it.
- → The script should accept different portfolios (not hardcoded to one example).
- → Print the raw API response and the extracted structured output separately.

→ Include in your README: what prompt approach you tried, what worked, what you changed and why.

Bonus (Optional)

→ Make the tone configurable: 'beginner', 'experienced', or 'expert' — and adjust the Claude prompt accordingly.

→ Add a second LLM call that critiques the first explanation for accuracy.

What We Are Evaluating

DIMENSION	WEIGHT	WHAT WE LOOK FOR
Prompt quality	14 pts	Is your prompt well-structured? Does it produce consistent, useful output?
Output quality	10 pts	Is the generated explanation actually good? Clear, accurate, actionable?
Code structure	6 pts	Clean separation of prompt logic, API call, and output parsing

---

04 THE OPEN PROBLEM

Task 04 **The Open Problem**	20 pts weight	Easy difficulty

No Specification — That Is The Point

Look at timecell.ai. Understand what the product does.

Now build something — anything — that you think would make it better, more useful, or more interesting. It does not need to be big. It could be:

→ A CLI tool that does something useful for portfolio tracking

→ A small script that answers a financial question you found interesting

→ A data visualisation of something relevant to wealth management

→ An experiment with AI that you think has product potential

→ Literally anything that shows how you think about the problem space

WHY THIS TASK EXISTS

The first engineering hire at a startup does not wait to be told what to build. This task has no right answer. We are looking for curiosity, initiative, and taste — not a correct solution.

What We Are Evaluating

DIMENSION	WEIGHT	WHAT WE LOOK FOR
Originality	8 pts	Did you think for yourself, or default to the obvious?
Execution	5 pts	Does it actually work?
Judgment	6 pts	Is this something worth building? Can you explain why?

05 SUBMISSION INSTRUCTIONS

Submit via email to sandeep@timecell.ai with the subject line:

```
Timecell Intern Test — [Your Name] — [College]
```

Your email must contain:

- → Link to your public GitHub repo (timecell-intern-<your-name>)
- → Link to your Loom / screen recording walkthrough (3–5 minutes)
- → One paragraph: what was the hardest part, and how did you approach it?

Overall Scoring Summary

TASK	MARKS	PRIMARY SKILL TESTED
Task 01 — Portfolio Risk Calculator	30	Python · Quantitative thinking
Task 02 — Live Market Data Fetch	20	APIs · Error handling · Clean code
Task 03 — AI-Powered Portfolio Explainer	30	LLM prompting · AI integration
Task 04 — The Open Problem	20	Initiative · Judgment · Curiosity
TOTAL	100	

A NOTE FROM SANDEEP

We built Timecell to run inside Claude Code — a terminal, not a dashboard. The engineers we want are the ones who are genuinely excited by that. If you find yourself going down a rabbit hole on one of these problems because it got interesting, that is a good sign. Good luck.